Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Modelación hardware/software con orientación a objetos (CE-5507)

Workshop 5:
Git Introduction

Professor:
Marco Hernandez Vasquez

Students:
Adriel Sebastián Chaves Salazar
Daniel Duarte Cordero

I Semester 2026

Contents

1 Investigation **3**

2 Evidence of Part 1: Repository Creation **3**
2.1 Repository Link and Collaborator Setup 3
2.2 Conflict Resolution 3
2.3 Branches Created 4
2.4 History of Changes 4

3 Reflection **5**

4 Evidence of the Exercise **5**
4.1 Branches Created 6
4.2 History of Changes 6

1 Investigation

Before starting the workshop, the following key concepts related to version control were reviewed [1,2]:

- **Version Control:** A system that records changes to a file or set of files over time, allowing specific versions to be recalled later. It enables multiple people to collaborate on the same project without overwriting each other's work [1].
- **Repository:** A storage location where a project's files and the entire history of changes are kept. It can be local (on a developer's machine) or remote (hosted on a platform such as GitHub) [1].
- **Commit:** A snapshot of the repository at a specific point in time. Each commit records what changed, who made the change, and when, along with a descriptive message [2].
- **Conflict (version control context):** A situation that arises when two collaborators make changes to the same part of a file independently, and the version control system cannot automatically determine which change to keep [1].
- **Branch:** A parallel line of development within a repository. Branches allow developers to work on features or fixes in isolation without affecting the main codebase, and can later be merged back [2].
- **Markdown file (.md):** A plain-text file using lightweight markup syntax that can be rendered into formatted text. Commonly used for documentation, README files, and project descriptions on platforms like GitHub [3].

2 Evidence of Part 1: Repository Creation

2.1 Repository Link and Collaborator Setup

The repository was created on GitHub by one of the team members and the second member was added as a collaborator. Both members then cloned the repository using GitHub Desktop [3].

The repository can be accessed at:
`https://github.com/Adriel23456/Workshop5_Modelado_AdrielChaves-DanielDuarte`

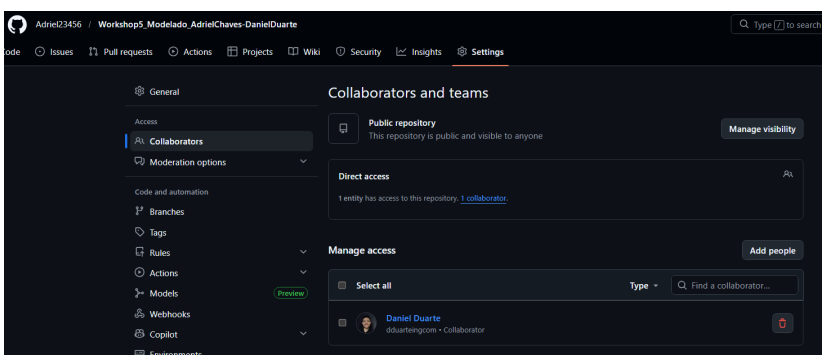

Figure 1: Repository created on GitHub with collaborator added

2.2 Conflict Resolution

Both team members edited the same file simultaneously without pulling first, which caused a merge conflict. The conflict was detected and resolved [1].

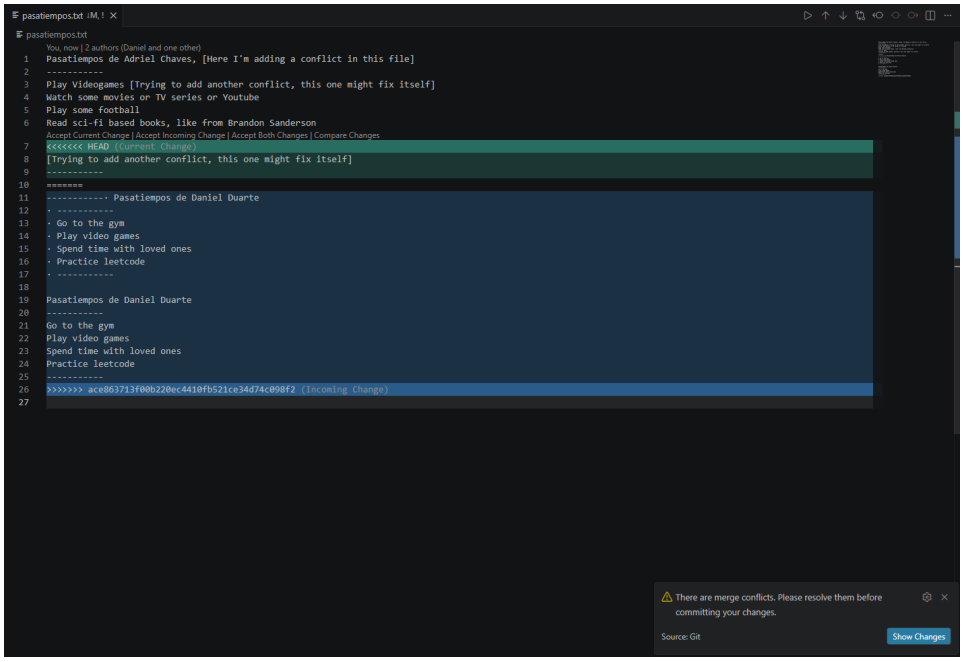

Figure 2: Merge conflict detected

2.3 Branches Created

A new branch named `version-alternativa` was created from the main branch. Changes were committed to this branch and later merged back into the main branch via a Pull Request [3].

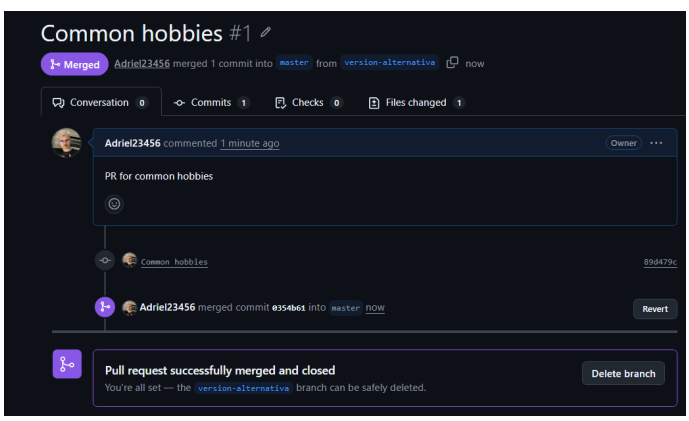

Figure 3: Pull Request created and merged into the main branch

2.4 History of Changes

The commit history was reviewed in both GitHub Desktop and on GitHub, showing each change, its author, and its timestamp.

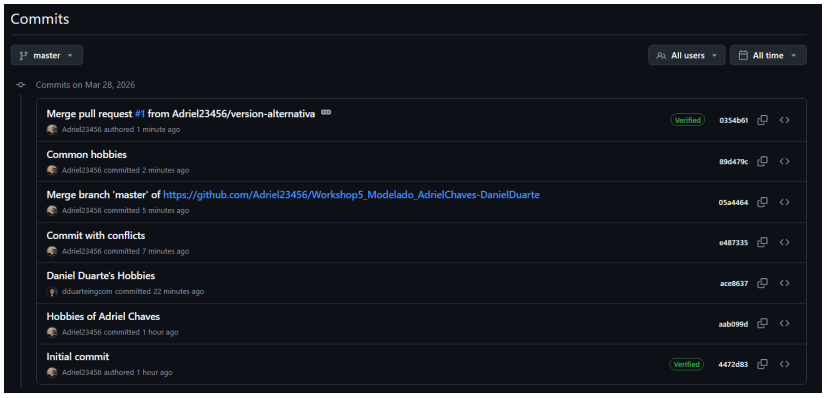

Figure 4: Full commit history showing both authors and all changes

3 Reflection

1. **What advantages does version control offer?**
   Version control allows teams to track every change made to a project, recover previous versions when errors are introduced, and collaborate without risk of overwriting each other's work. It also provides a complete and auditable history of the project's evolution [1].

2. **What difficulties were encountered during the process?**
   The main difficulty was understanding how to resolve merge conflicts. When both members edited the same line simultaneously, reconciling the two versions required careful reading of both changes to produce a consistent final result.

3. **What happened during the conflict and how was it resolved?**
   The conflict occurred because both members modified the first line of `pasatiempos.txt` without pulling the latest version first. GitHub Desktop flagged the conflict and provided a visual tool to compare both versions. The final file was manually edited to merge both contributions correctly, then committed and pushed.

4. **What is the utility of branches in a real project?**
   Branches allow developers to work on new features or experimental changes in isolation, without affecting the stable main branch. Once the work is validated, it can be merged cleanly. This is essential in professional workflows where the main branch must always remain in a working state [2].

4 Evidence of the Exercise

The exercise consisted of proposing a simple interactive system described in a Markdown file. Both members contributed changes in parallel, including the use of at least one branch and a merge process.

4.1 Branches Created

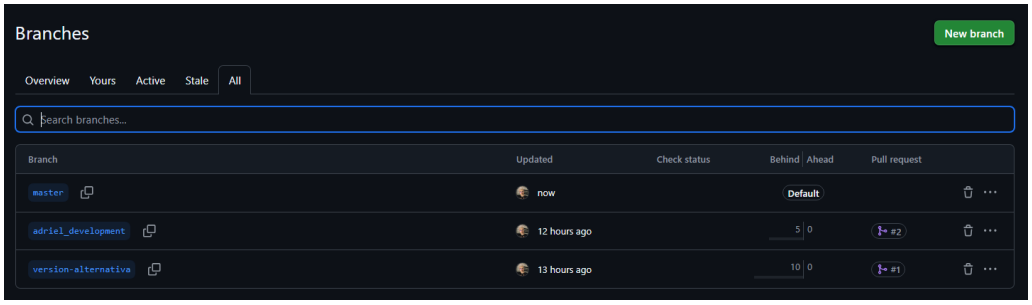

Figure 5: Branch created during the exercise development

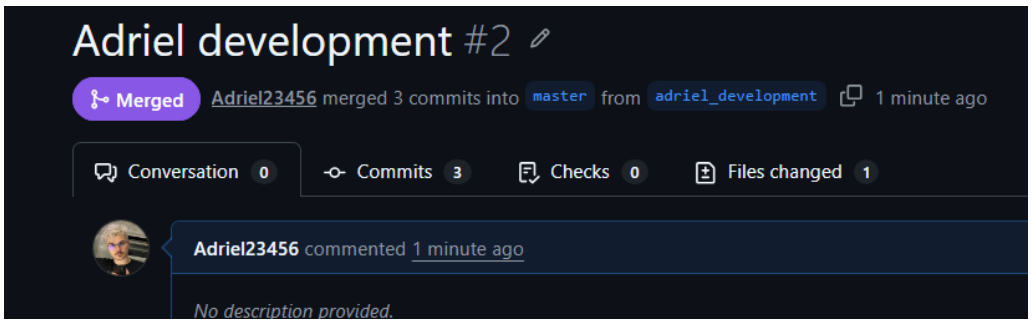

Figure 6: Merge of exercise branch back into the main branch

4.2 History of Changes

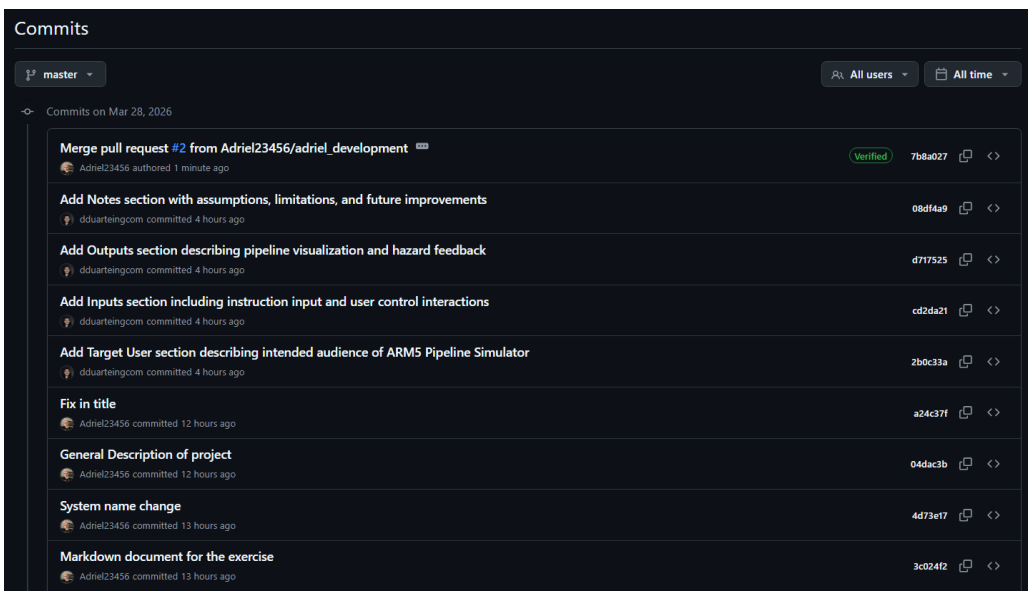

Figure 7: Commit history for the exercise showing contributions from both team members

References

[1] S. Chacon and B. Straub, *Pro Git*, 2nd ed. New York: Apress, 2014. [Online]. Available: https://git-scm.com/book/en/v2

[2] J. Loeliger and M. McCullough, *Version Control with Git*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2012.

[3] GitHub, Inc., "Github documentation," https://docs.github.com, 2024, accessed: 2025.